CS5224 Cloud Computing, Lab 2

Cloud Services

Name: Ishan Agarwal
Student ID: A0284673R
Cloud provider for Part 2: AWS, region ap-southeast-1 (Singapore)
Source code: https://github.com/ishan-agarwal-05/cs5224-lab2

Part 1: Cloud Services

(a) Healthcare analytics SaaS for multiple hospitals

Deployment model. I would use a hybrid cloud made up of three deployment models.

- **Private cloud at each hospital.** Identifiable patient records stay in each hospital's own private cloud. This gives the hospital the highest level of control over its most sensitive data.
- **Community cloud for the shared SaaS.** The analytics SaaS runs on a community cloud that only the participating hospitals use. The hospitals share the same regulatory requirements, and healthcare is one of the typical community cloud examples from the lectures. It costs less than every hospital building its own private platform, access is limited to member hospitals, and there is one agreed governance structure for managing it.
- **Public cloud for extra capacity.** The public cloud is only used to absorb demand spikes, through cloud bursting (explained below).

Before any data leaves a hospital, an export service removes the fields the analytics do not need and replaces patient identifiers with pseudonyms. The table that maps a pseudonym back to a real patient never leaves the hospital. This means the community cloud only holds pseudonymised data, and only pseudonymised data can ever burst to the public cloud. All of the clouds would be located in the same country, or on a sovereign cloud, so the data stays within national borders.

The SaaS is multi-tenant, with each hospital as one tenant. Tenant isolation makes sure a hospital cannot see another hospital's data or configuration, and I would use data tier isolation by giving each hospital its own database instead of shared tables. Data is encrypted in transit and at rest, and every access is logged so that a cloud auditor can check compliance. National reports that combine hospitals only use aggregated data.

Scaling architecture. I would combine three of the resource organisation architectures from the lectures.

- **Workload distribution with service load balancing.** The web and API services are stateless and replicated on several virtual servers, with an independent load balancer in front (for example an AWS Application Load Balancer). The load balancer spreads requests across the copies and only sends traffic to healthy ones.
- **Dynamic scalability using dynamic horizontal scaling.** An automated scaling listener watches request count and CPU usage. When an outbreak pushes these past a set threshold, more copies of the service are added from the resource pool, and they are removed when demand drops. I prefer horizontal scaling here because one server can only be made so big. Heavy analytics jobs go on a queue and run on a separate worker pool (for example a Spark cluster) that scales the

same way, so a spike in report requests does not slow down the normal dashboards. The database is harder to scale out, so for it I would use dynamic vertical scaling and give the database server more CPU and memory when it is under load.

- **Cloud bursting.** If the community cloud reaches its capacity threshold during a large outbreak, the automated scaling listener bursts out to redundant copies of the analytics service that are already deployed on the public cloud. When demand falls they burst back in and the public cloud resources are released. Resource replication keeps the state on both sides in sync.

To scale nationally, each new hospital joins as another tenant and the data is partitioned by hospital and region. The platform can be deployed in more regions if needed, with a second region kept for disaster recovery.

The downside is that a hybrid of three clouds is harder to operate and needs reliable network links between them. I still think it is the right choice. A purely private setup could only handle outbreak spikes by buying a lot of hardware that sits idle most of the time, and putting all the raw patient data on a public cloud would make the governance requirement much harder to meet.

(b) Modernising a bank's legacy application

Service model. I would use PaaS. In the lectures PaaS is the model where the consumer deploys its own application onto a platform that already has the database, frameworks and middleware ready. That matches what the bank wants, which is to modernise its own application rather than replace it with someone else's. With a managed platform (for example AWS Elastic Beanstalk or ECS on Fargate) and a managed database (for example Amazon RDS), the provider handles the servers, operating system patching, scaling and backups. The bank's developers can spend their time breaking the old application into smaller services, and the bank still controls its own code, data, user access and configuration.

I compared it with the other options using the pros and cons from the lectures. Staying on-premise gives the most flexibility but has a long time-to-market, which is already the problem with the legacy system. IaaS removes the need to buy hardware, but the bank would still manage and patch its own operating systems and middleware, and privacy is listed as a concern. IaaS could still be used for a short time for legacy parts that cannot be changed yet. SaaS has the lowest flexibility and is for using a provider's application, not running the bank's own. The main drawback of PaaS is vendor lock-in. I would reduce it by packaging the services in containers and using standard databases, so the bank could move to another provider later.

Deployment model. I would use a hybrid cloud, with the sensitive data on a private cloud and the modernised application on a public cloud PaaS. This is the same split the lectures use as the example for hybrid cloud. The most sensitive data, such as customer account records and encryption keys, stays in the bank's on-site private cloud. There the bank has full control and can show the regulator exactly where the data is. The new application services run on the public cloud PaaS inside a virtual private cloud, so they are isolated from other customers, and they connect to the private side over a dedicated link (for example AWS Direct Connect). Fully public would put the sensitive data outside the bank's control. Fully private would mean the bank has to build and run its own platform, which loses most of the benefit of PaaS.

For compliance I would choose a PaaS provider whose services are physically hosted in the same country as the bank. The lectures show this as a way to meet legal requirements for data storage. Data would be encrypted with keys the bank manages, access would use least privilege and MFA,

and all access would be logged so an independent cloud auditor can check the setup against the regulator's rules (in Singapore, the MAS Technology Risk Management Guidelines). I would migrate one part of the legacy system at a time and keep the old system running until each new part has been tested, so there is always a way to roll back.

Part 2: Data Analyse It!

Overview

I used AWS for this part. The dataset and my PySpark script are stored in an S3 bucket, and the analysis runs as an AWS Glue Spark job. The backend is written in Python and the frontend is a plain HTML and JavaScript page. Figure 1 shows how the parts connect. All of the code is on GitHub at https://github.com/ishan-agarwal-05/cs5224-lab2, and the README there explains how to run it.

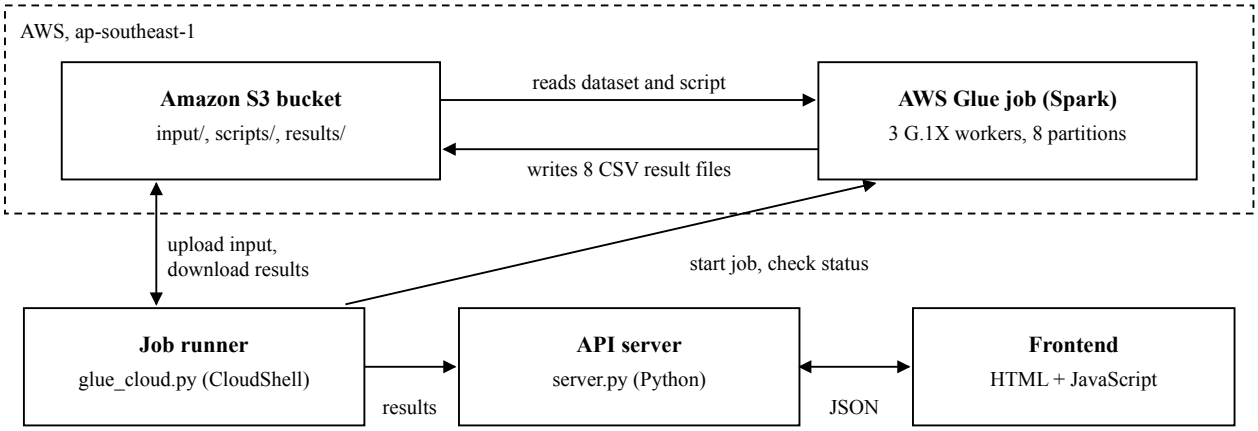


Figure 1: How the frontend, backend and Glue job fit together.

The steps for one run are:

1. The job runner uploads `twitter_combined.txt` and `spark_job.py` to the S3 bucket and creates the Glue job.
2. It starts the Glue job. Glue gives the job three workers and runs the Spark script on the dataset in S3.
3. The Spark job writes the counts back to S3 as 8 CSV files, plus one small JSON file with totals.
4. The job runner downloads the result files and checks them. The API server loads them and the frontend asks the API for the numbers.

Cleaning the dataset

Each line `A B` means A follows B, so A gets one followee and B gets one follower. The spec says the file has not been cleaned, so I checked it before writing the job. There are no blank or malformed lines, but there are a lot of repeated lines and a few lines where a user follows themselves. I made these decisions:

- If the same line `A B` appears more than once, it is counted once. `A B` and `B A` are different relationships and both count.

- Lines where A and B are the same user are dropped, because following yourself is not a real follow. None of these 22 lines involve the two users the lab asks about, so this choice does not change their answers.
- User IDs are kept as strings so very long IDs cannot lose precision, and leading zeros are stripped so `0123` and `123` count as the same user.

Item	Count
Lines in the file	2,426,143
Blank or malformed lines	0
Self-follow lines removed	22
Duplicate lines removed	657,986
Unique follow relationships left	1,768,135
Distinct users	81,306

Results

These are the results for the two users in the spec, in the required format:

```
214328887 has 180 followers
214328887 has 214 followees
107830991 has 180 followers
107830991 has 157 followees
```

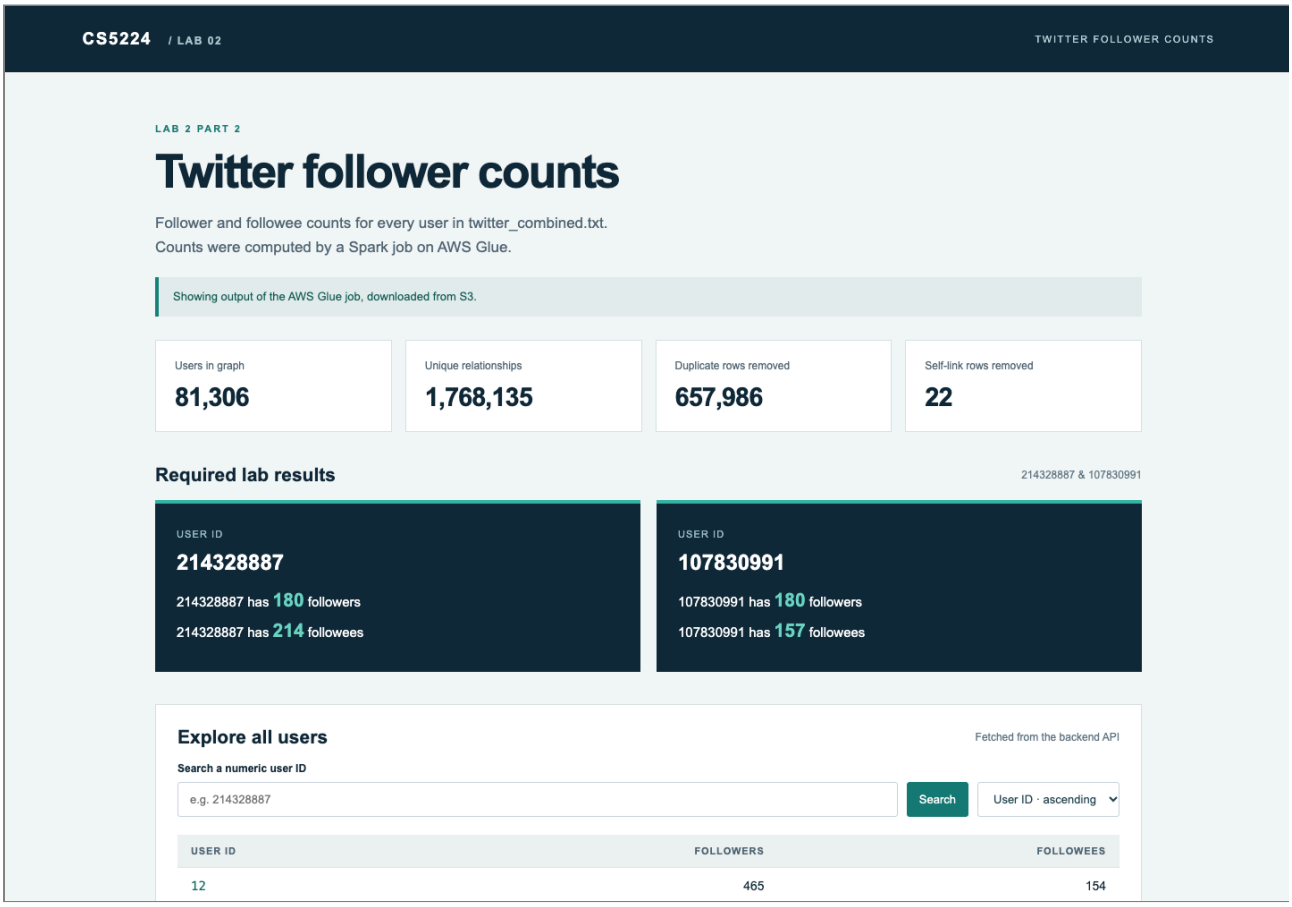


Figure 2: The frontend showing both users. The numbers come from the backend API, which serves the output of the Glue job.

This is the raw response from the backend endpoint that the frontend calls for these two users:

```
GET /api/users?ids=214328887,107830991

{"users": [{"user_id": "214328887", "followers": 180, "followees": 214},
           {"user_id": "107830991", "followers": 180, "followees": 157}],
 "missing": []}
```

1. Front-end

The frontend is one HTML page with CSS and plain JavaScript. The backend serves the page, and the script uses `fetch` to call the backend's JSON API, for example `/api/users?ids=214328887,107830991`. It shows "UserID has N followers/followees" for both users. It also has a searchable, paged table of all users. (49 words)

2. Back-end

The backend is written in Python and has two parts. The job runner (`glue_cloud.py`, run in AWS CloudShell with boto3) uploads the dataset and Spark script to S3, creates and starts the AWS Glue job, checks its status until it succeeds, then downloads the result files from S3 and verifies them. The API server (`server.py`, Python standard library) loads these results, checks that the totals add up, keeps the counts in memory, and serves them as JSON through `/api/users` and `/api/summary`. It also serves the frontend files. (87 words)

3. Data analysis component

For the analysis I used AWS Glue, which runs Apache Spark jobs without me having to set up a cluster. I first tried EMR Serverless, but my account is on the AWS free plan and the EMR API returned `SubscriptionRequiredException`. Glue was available on the same account and also runs Spark, so I switched to it. The docstring at the top of my script still mentions EMR Serverless from that first attempt.

The job settings were Glue 5.0 (Spark 3.5.4), 3 workers of type G.1X (4 vCPU and 16 GB memory each), 8 partitions, a 10 minute timeout and no automatic retries. The input path and output path in S3 are passed to the script as job arguments. The run on 11 October 2026 succeeded on the first attempt and took 115 seconds of execution time. Glue reported 347 DPU-seconds, which at the Singapore rate of US$0.44 per DPU-hour is about US$0.04.

The full script is in Appendix A. It works in the same map and reduce style as MapReduce:

1. **Read and split.** Spark reads the file from S3 one line at a time and spreads the lines over 8 partitions, so the workers process them in parallel.
2. **Parse and clean** (`clean_edges`). Each line is checked with a regular expression and split into a source and destination ID. Malformed lines and self-follows are filtered out, then `dropDuplicates` removes repeated pairs. This step shuffles the data across workers, so a duplicate is removed even if its copies were in different partitions.
3. **Map** (`count_users`). For each unique edge `A B` the job emits two records: `(A, followers=0, followees=1)` and `(B, followers=1, followees=0)`.
4. **Reduce.** The records are grouped by user ID and the two columns are summed. A user who only appears on one side of every edge still gets a row, with 0 on the other side.

5. **Write.** The counts are written back to S3 as 8 CSV files, one per partition, plus a JSON file with the cleaning totals.

The screenshots below show the job running on AWS.

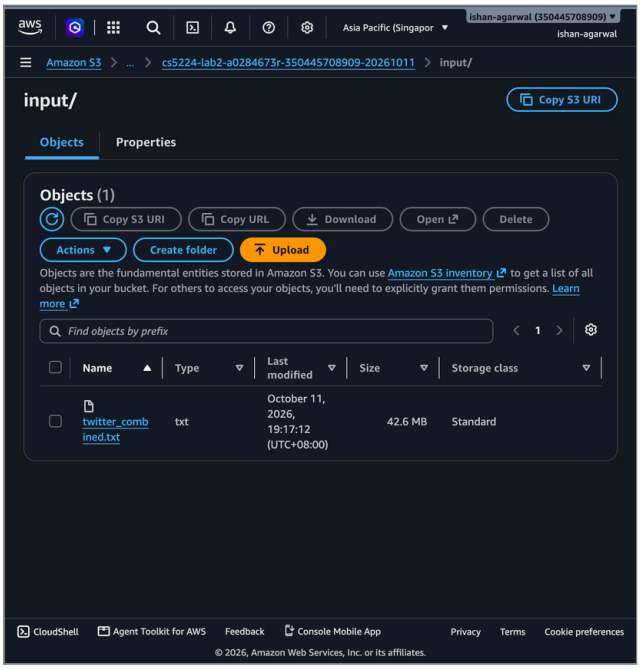


Figure 3: Dataset uploaded to `input/` in the S3 bucket.

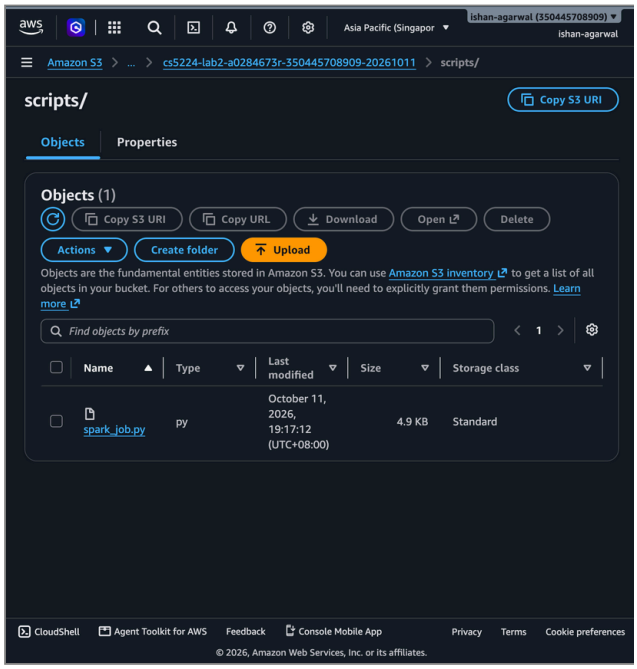


Figure 4: Spark script uploaded to `scripts/` in the same bucket.

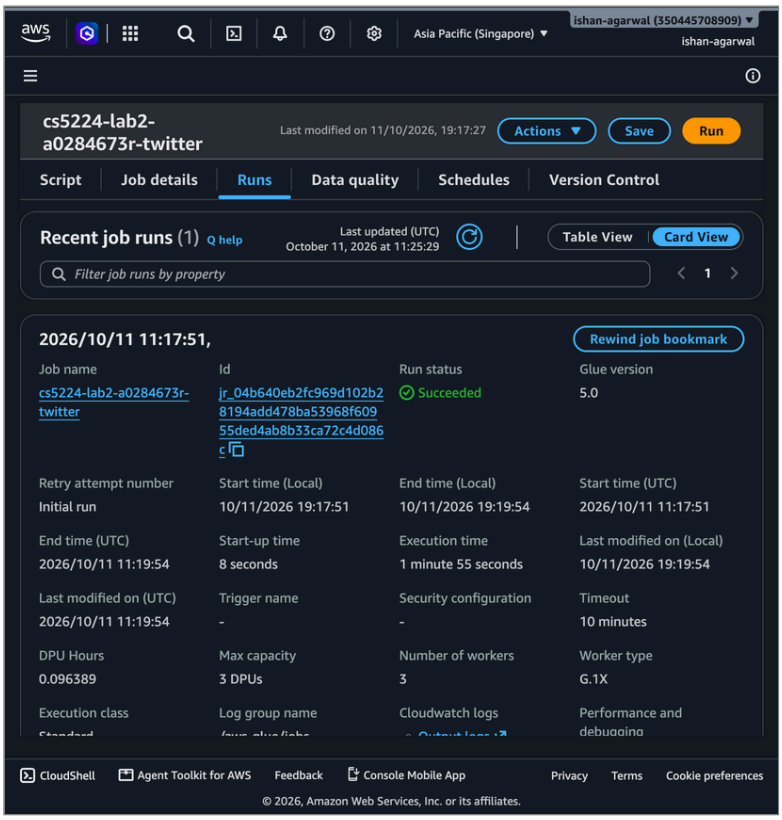


Figure 5: The Glue job run. It succeeded on the initial run with Glue 5.0, 3 G.1X workers and an execution time of 1 minute 55 seconds.

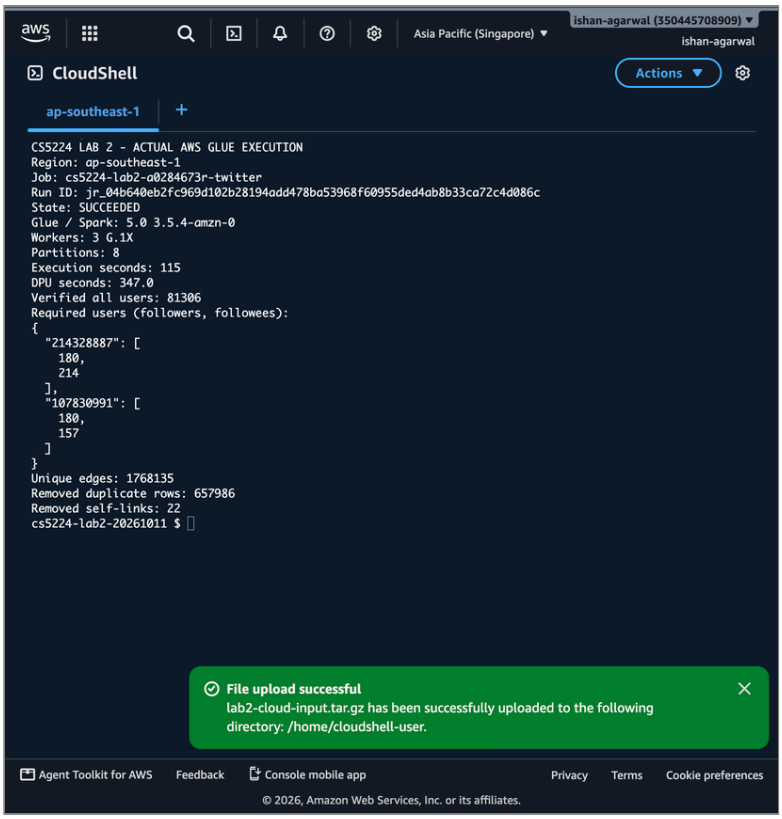


Figure 6: Output of my job runner in CloudShell after the run finished, showing the run ID, settings and the counts for the two users.

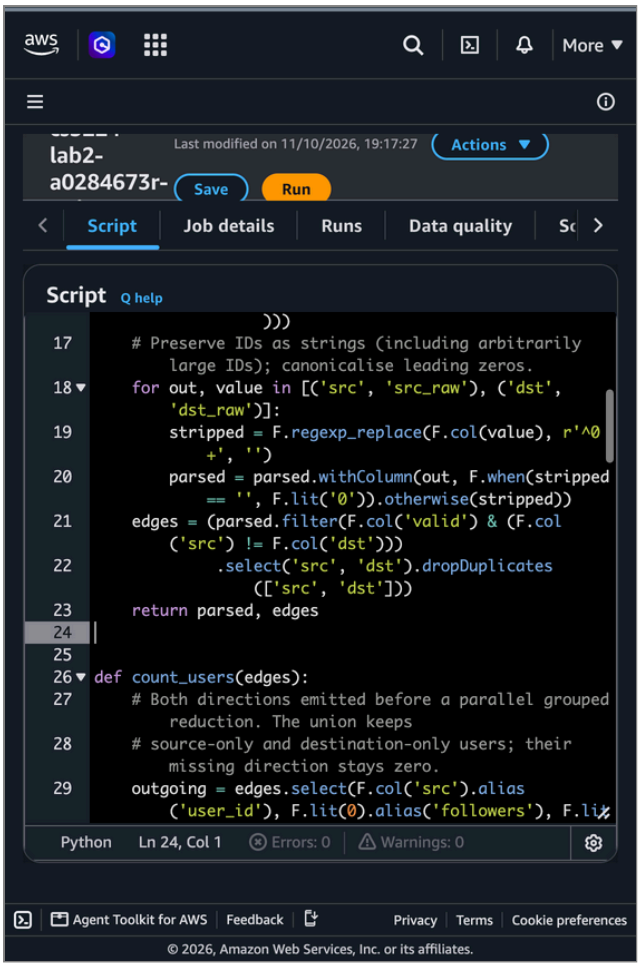


Figure 7: The script in the Glue editor, showing the cleaning and deduplication step.

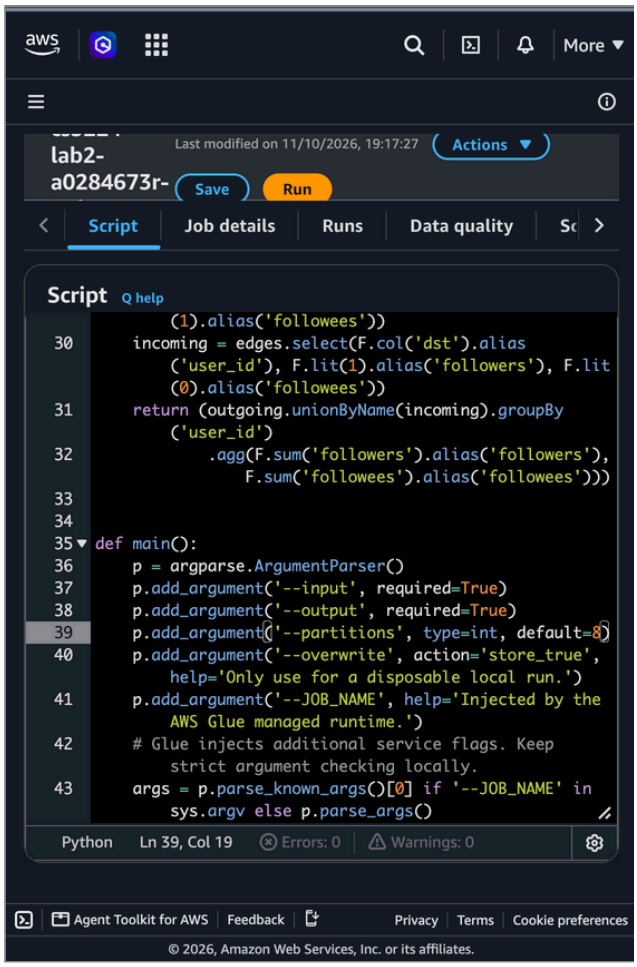


Figure 8: The map and reduce step, where both directions are combined and summed per user.

Checking the results

Before running on AWS I tested the same script with a local Spark install (`local[4]`, 8 partitions). I also wrote a short plain Python version (`reference.py`) that uses a set to remove duplicates and a counter to count followers and followees. The Glue output matched it for all 81,306 users. As a sanity check, the total number of followers and the total number of followees across all users are both 1,768,135, which is the number of unique edges. That has to be true because every edge adds exactly one follower and one followee.

If I was doing this for real I would also host the API server on AWS (for example on Lambda behind API Gateway, reading the results straight from S3) instead of running it on my laptop. I left that out because the spec only asks for the analysis to run on the cloud.

AI use declaration

I worked on this lab myself, with guidance and help from AI tools along the way. I used Manus AI to help me understand the requirements, draft the Part 1 answers, write and debug the frontend, backend and Spark code, and set up, run and check the Glue job in my AWS account. At the end I used Claude to check and grade my report against the lab specification, rework the layout and edit the wording. I am responsible for everything in this submission.

References

1. A. Gopalan, CS5224 lecture slides, Lecture 2 (Concepts and Models), Lecture 3 (Cloud Architectures) and Lecture 5 (Virtualisation and Multitenancy), NUS, 2026.
2. T. Erl, Z. Mahmood and R. Puttini, Cloud Computing: Concepts, Technology & Architecture, Prentice Hall, 2013. Chapters 4 and 11.
3. NIST SP 800-145, The NIST Definition of Cloud Computing. https://csrc.nist.gov/pubs/sp/800/145/final
4. Stanford SNAP, Social circles: Twitter. https://snap.stanford.edu/data/egonets-Twitter.html
5. AWS Glue Developer Guide, Configuring job properties for Spark jobs. https://docs.aws.amazon.com/glue/latest/dg/add-job.html
6. AWS Glue pricing. https://aws.amazon.com/glue/pricing/

Appendix A: Spark job (`analysis/spark_job.py`)

This is the script that was uploaded to S3 and run by the Glue job. The backend, frontend and job runner code are in the GitHub repository linked above.

```
"""Run unchanged with local spark-submit or Amazon EMR Serverless.

A line 'A B' means A follows B. Duplicates must not inflate graph degrees.
Output is partitioned CSV plus a one-row metadata JSON directory.
"""
import argparse
import sys
from pyspark.sql import SparkSession, functions as F
from pyspark.storagelevel import StorageLevel


def clean_edges(raw):
    parsed = (raw.select(F.trim('value').alias('line'))
              .withColumn('valid', F.col('line').rlike(r'^[0-9]+\s+[0-9]+$'))
              .withColumn('src_raw', F.regexp_extract('line', r'^([0-9]+)\s+([0-9]+)$', 1))
              .withColumn('dst_raw', F.regexp_extract('line', r'^([0-9]+)\s+([0-9]+)$', 2)))
    # Preserve IDs as strings (including arbitrarily large IDs); canonicalise leading zeros.
    for out, value in [('src', 'src_raw'), ('dst', 'dst_raw')]:
        stripped = F.regexp_replace(F.col(value), r'^0+', '')
        parsed = parsed.withColumn(out, F.when(stripped == '',
F.lit('0')).otherwise(stripped))
    edges = (parsed.filter(F.col('valid') & (F.col('src') != F.col('dst')))
             .select('src', 'dst').dropDuplicates(['src', 'dst']))
    return parsed, edges


def count_users(edges):
    # Both directions emitted before a parallel grouped reduction. The union keeps
    # source-only and destination-only users; their missing direction stays zero.
    outgoing = edges.select(F.col('src').alias('user_id'), F.lit(0).alias('followers'),
F.lit(1).alias('followees'))
    incoming = edges.select(F.col('dst').alias('user_id'), F.lit(1).alias('followers'),
F.lit(0).alias('followees'))
    return (outgoing.unionByName(incoming).groupBy('user_id')
            .agg(F.sum('followers').alias('followers'),
F.sum('followees').alias('followees')))


def main():
    p = argparse.ArgumentParser()
    p.add_argument('--input', required=True)
    p.add_argument('--output', required=True)
    p.add_argument('--partitions', type=int, default=8)
    p.add_argument('--overwrite', action='store_true', help='Only use for a disposable local
run.')
    p.add_argument('--JOB_NAME', help='Injected by the AWS Glue managed runtime.')
    # Glue injects additional service flags. Keep strict argument checking locally.
    args = p.parse_known_args()[0] if '--JOB_NAME' in sys.argv else p.parse_args()
    if args.partitions < 2:
        p.error('Use at least two partitions to demonstrate parallel processing.')
    spark = SparkSession.builder.appName('CS5224-Twitter-Degrees').getOrCreate()
    spark.sparkContext.setLogLevel('WARN')
    spark.conf.set('spark.sql.shuffle.partitions', args.partitions)
    spark.conf.set('spark.sql.adaptive.enabled', 'false')
    raw = spark.read.text(args.input).repartition(args.partitions)
    parsed, edges = clean_edges(raw)
    parsed.persist(StorageLevel.MEMORY_AND_DISK)
    edges.persist(StorageLevel.MEMORY_AND_DISK)
    counts = count_users(edges).persist(StorageLevel.MEMORY_AND_DISK)
    metrics = parsed.agg(
        F.count('*').alias('raw_lines'),
        F.sum(F.when(F.col('line') == '', 1).otherwise(0)).alias('blank_lines'),
        F.sum(F.when((F.col('line') != '') & ~F.col('valid'),
```

```
1).otherwise(0)).alias('malformed_lines'),
        F.sum(F.when(F.col('valid') & (F.col('src') == F.col('dst')),
1).otherwise(0)).alias('self_loop_lines'),
        F.sum(F.when(F.col('valid') & (F.col('src') != F.col('dst')),
1).otherwise(0)).alias('valid_nonself_lines'),
    ).first().asDict()
    metrics = {k: int(v or 0) for k, v in metrics.items()}
    metrics['unique_edges'] = edges.count()
    metrics['duplicate_edges_removed'] = metrics['valid_nonself_lines'] -
metrics['unique_edges']
    totals = counts.agg(F.count('*').alias('unique_users'),
F.sum('followers').alias('total_followers'),
                        F.sum('followees').alias('total_followees')).first().asDict()
    metrics.update({k: int(v or 0) for k, v in totals.items()})
    assert metrics['total_followers'] == metrics['total_followees'] == metrics['unique_edges']
    metrics.update(engine='Apache Spark', spark_version=spark.version,
                   execution_mode='local' if spark.sparkContext.master.startswith('local')
else 'cloud/distributed',
                   master=spark.sparkContext.master, partitions=args.partitions,
input_uri=args.input,
                   cleaning_policy='Numeric directed edges; canonical IDs; drop blanks,
malformed rows, self-links; deduplicate (source,target).')
    mode = 'overwrite' if args.overwrite else 'errorifexists'
    counts.repartition(args.partitions).write.mode(mode).option('header',
True).csv(args.output.rstrip('/') + '/users')
    spark.createDataFrame([metrics]).coalesce(1).write.mode(mode).json(args.output.rstrip('/')
+ '/metadata')
    print('AUDIT:', metrics)
    print('REQUIRED USERS:')
    counts.filter(F.col('user_id').isin('214328887',
'107830991')).orderBy('user_id').show(truncate=False)
    counts.unpersist(); edges.unpersist(); parsed.unpersist()
    spark.stop()


if __name__ == '__main__':
    main()
```